

Sentinelleai — Audit Report

ERC20TokenAutoLiquidity.sol · cbb823bb-4fad-48c0-850b-08252a41255c · 2026-05-22 17 h 52 min 08 s

CAUTION 42 / 100 9 consolidated findings

ERC20TokenAutoLiquidity presents a significant centralization risk (SC01:2026 Access Control) confirmed by three independent agents and corroborated by historical exploit patterns: a single EOA owner controls pause/unpause, treasury rotation, and tax-exemption grants with no timelock or multisig enforcement. A compromised or malicious owner can redirect all tax revenue, freeze all transfers permanently via pause-then-renounce (SC10:2026 Denial of Service), and grant privileged tax-free trading to arbitrary addresses. Secondary issues include misleading 'liquidity burn' semantics (tokens sent to 0xdead rather than supply-reduced via _burn, SC03 Logic Errors), stale tax exemptions on rotated treasury addresses (SC03), fee-on-transfer integration hazards for naive third-party protocols (SC03), and a missing tokenURI update function despite an update event. The contract is functional for its stated purpose but is unsuitable for deployment with a single-EOA owner at any meaningful value threshold.

Severity breakdown

1 CRITICAL 1 HIGH 4 MEDIUM 2 LOW 1 INFO

Consolidated findings

CRITICAL CVSS 9.1 SC01:2026 - Access Control

Single-EOA owner controls all privileged functions with no multisig or timelock

The owner (potentially a single EOA) has immediate, unrestricted control over `setTaxExempt()`, `setTreasuryAddress()`, `pause()`, and `unpause()`. A compromised or malicious owner can: redirect all 2% tax revenue to an attacker-controlled treasury, grant tax-free trading to arbitrary addresses while other users pay the full tax, and freeze all transfers permanently. This matches the exact pattern of the Wasabi (\$25M, Apr 2026) and Drift (\$285M, Apr 2026) exploits where single admin keys were compromised or socially engineered. Ownable2Step mitigates accidental ownership transfer but does not protect against key compromise or insider threat.

`constructor`, `setTaxExempt()`, `setTreasuryAddress()`, `pause()`, `unpause()`

→ Replace Ownable2Step with a 3-of-5 multisig as the owner. Enforce a minimum 48-hour timelock on `setTreasuryAddress()`, `setTaxExempt()`, and `pause()`. Separate roles for pause guardian, treasury manager, and exemption manager. Emit scheduled-operation events so users can exit before critical parameter changes take effect.

Confirmed by: attacker, historian, formalist

HIGH CVSS 7.5 SC10:2026 - Denial of Service

Owner can permanently freeze all transfers by pausing then renouncing ownership

ERC20Pausable is enforced inside `_update()`, so a paused contract blocks all transfers, burns, and mints. Because Ownable (via Ownable2Step) exposes `renounceOwnership()`, an owner or attacker controlling the owner key can call `pause()` followed by `renounceOwnership()`, permanently bricking the token with no recovery path. This irreversibly prevents all user exits, DEX trading, and CEX operations.

`pause()`, `unpause()`, `_update()` override, `renounceOwnership()` inherited from Ownable

→ Override `renounceOwnership()` to revert if the contract is paused. Consider a separate pause guardian role that can pause but cannot renounce ownership. Add an emergency unpause path controlled by a timelocked multisig or governance contract.

Confirmed by: attacker

MEDIUM CVSS 5.3 SC03:2026 - Logic Errors

Old treasury address retains permanent tax exemption after treasury rotation

setTreasuryAddress() grants tax exemption to the new treasury but does not revoke it from the old treasury. Over time, every historical treasury address accumulates a permanent tax-free status. If an old treasury key is later compromised or transferred, the attacker can route tokens through it in both directions (isTaxExempt[from] and isTaxExempt[to]) without paying the 2% tax, creating privileged trading paths invisible to users.

```
setTreasuryAddress(address newTreasury)
```

→ Add isTaxExempt[oldTreasury] = false; before setting the new treasury, unless the old address requires exemption for another documented reason. Emit a TaxExemptionUpdated event for the revocation.

Confirmed by: attacker, developer

MEDIUM CVSS 5.0 SC01:2026 - Access Control

Arbitrary tax exemptions allow privileged trading and bypass of advertised tokenomics

The owner can exempt any address from the 2% transfer tax at any time without delay. Because _update() skips tax if either sender or recipient is exempt, an exempt address can both send and receive without tax. This enables a privileged actor to trade under different rules than ordinary users, undermining the advertised tokenomics and creating unfair execution.

```
mapping isTaxExempt; setTaxExempt(); _update() tax skip condition
```

→ Constrain tax exemptions to a small, immutable or timelocked allowlist of known system addresses (e.g., AMM routers, staking contracts). Require delayed governance for exemption changes. Publish all privileged addresses publicly and avoid granting exemptions to EOAs.

Confirmed by: attacker

MEDIUM CVSS 4.8 SC03:2026 - Logic Errors

Misleading 'liquidity burn' sends tokens to Oxddead instead of reducing total supply

The constructor labels the operation as burning liquidity and emits a LiquidityBurned event, but it only transfers ERC20 tokens to address(0xdead). This does not reduce totalSupply (which requires transfer to address(0)), and it does not prove any AMM LP tokens were created or burned. Users may interpret initialLiquidityBurned and the LiquidityBurned event as evidence of real liquidity lock or reduced circulating supply, which is not guaranteed by this contract. This constitutes a misleading representation that could be used in rug-pull marketing.

```
constructor liquidityToBurn block; initialLiquidityBurned; LiquidityBurned
event
```

→ If the goal is supply reduction, use `_burn(initialOwner, liquidityToBurn)` to transfer to `address(0)` and reduce `totalSupply`. If the goal is locking tokens, rename the variable and event to reflect 'locked' rather than 'burned'. If the goal is LP token burn, require and verify LP token transfer to a burn address after liquidity creation. Add clear public documentation distinguishing circulating supply from dead-address balance.

Confirmed by: attacker, developer

MEDIUM CVSS 4.3 SC03:2026 - Logic Errors

Fee-on-transfer semantics can break naive third-party protocol integrations

`transfer()` and `transferFrom()` report success while the recipient receives amount minus the 2% tax whenever neither party is exempt. Many vaults, bridges, staking contracts, and OTC escrows incorrectly credit users by the requested transfer amount rather than the actual balance delta. A naive vault crediting deposits by the supplied amount rather than `balanceBefore/balanceAfter` can be drained by attackers exploiting the discrepancy between credited shares and actual token holdings.

```
_update() taxable transfer path; transfer()/transferFrom() inherited ERC20
behavior
```

→ Clearly document fee-on-transfer semantics in the contract NatSpec and README. Provide integration guidance requiring balance-delta accounting (`balanceBefore/balanceAfter` pattern). Consider adding a ERC20 interface marker or custom interface indicating fee-on-transfer behavior. Avoid representing this token as a standard no-fee ERC20 to external protocols.

Confirmed by: attacker

LOW CVSS 2.6 SC03:2026 - Logic Errors

Transfer tax can be bypassed via dust transfers below integer division rounding threshold

The tax calculation uses integer division: $\text{taxAmount} = \text{amount} * 200 / 10000$. Any transfer amount below 50 smallest token units produces $\text{taxAmount} == 0$. An attacker can split large transfers into chunks of 49 wei-equivalent units to avoid the 2% tax entirely, at the cost of additional gas. This is economically limited for 18-decimal tokens on high-gas chains but may be relevant for low-value tokens or chains with low gas costs.

`calculateTax(); _update()` tax calculation

→ If exact tax collection is required, enforce a minimum tax of 1 smallest unit for any nonzero taxable transfer above a configurable minimum amount, or reject taxable transfers below a threshold. Alternatively, document explicitly that dust transfers are tax-free by design.

Confirmed by: [attacker](#)

LOW CVSS 2.1 SC03:2026 - Logic Errors

Missing tokenURI update function despite MetadataUpdated event definition

The contract defines and emits a `MetadataUpdated` event in the constructor but provides no function to update `_tokenURI` after deployment. This creates an inconsistency: the event implies metadata is updatable, but the state variable is effectively immutable post-deployment.

`_tokenURI` variable; `MetadataUpdated` event; constructor

→ Either add a `setTokenURI(string memory newURI)` external `onlyOwner` function to allow metadata updates and emit `MetadataUpdated`, or remove the `MetadataUpdated` event if metadata is intended to be immutable post-deployment.

Confirmed by: [developer](#)

INFO CVSS 0.0 SC03:2026 - Logic Errors

Treasury address validation is correctly enforced but old treasury exemption cleanup is manual

The contract correctly validates that `treasuryAddress != address(0)` in both the constructor and `setTreasuryAddress()`. The formalist agent flagged this as an invariant to preserve, which is already satisfied by the existing `require` statements. No code change is needed for the zero-address check itself; the related stale-exemption issue is covered separately.

constructor, `setTreasuryAddress()`

→ No action required for zero-address validation. See the stale treasury exemption finding for the related cleanup recommendation.

Confirmed by: [formalist](#)

Generated by Sentinelleai on 2026-05-22T21:55:41.784Z. Audit ID: cbb823bb-4fad-48c0-850b-08252a41255c.

This report combines deterministic regex+AST sentinels, optional Slither/Mythril static analysis, and a 6-model adversarial LLM panel consolidated by a Judge agent.